control over output levels through a mixed initiative graphic tool that offers the ability to interact with the underlying WFC solver in various ways [17].

Of note is Lucas and Volz's straight forward application of counting tile frequency and measuring the Kullback-Leibler divergence to attempt to get a better understanding of how biased the resulting generated map is [18]. Karth and Smith use a vector-quantized variational auto encoder (VQ-VAE) to create reduced tile domain maps which can then be input into WFC to produce novel results [10].

Though automatic tile constraint creation and constraint based tiling generation are distinct ideas, they are often bundled together as manually creating tile constraints can be labor intensive and the resulting tile constraints are an input requirement to constraint based tiling generation problems. Gumin's *Wave Function Collapse* project highlighted an automatic tile constraint creation from exemplar scenes [5]. Many resources exist to explain automatic tile constraint creation in detail but of note are Sherrat's summary in [19] and an introduction by Newgas in [20]. We briefly go over automatic tile constraint generation in more detail later in this paper (Section 4).

3. Algorithm

3.1. Punch Out Model Synthesis

Punch Out Model Synthesis (POMS) works to progressively resolve chosen blocks within a larger grid. Blocks that fully resolve are then saved back into the larger grid. Should blocks not be able to fully resolve, portions of the grid are reverted back to an indeterminate state, depending on the mode of failure.

Algorithm 1 Punch Out Model Synthesis

```
Input,Output: Grid G,
Input: Block Choice Scheduler, BCS,
Input: Erosion Choice Scheduler, ECS,
Input: Block Solver Algorithm, A
Set G to a fully indeterminate state
while G not fully resolved do
    Choose block B in G by querying BCS
    Initialize B to indeterminate state
    From G, set and pin B edges not on G boundary
    Apply initial setup restrictions to B
    Run A(B)                    ▷ Attempt to resolve B with A
    if initial arc consistency is impossible for B then
        Set B region to indeterminate state in G
    else if A(B) fails to find a full resolution in B then
        Erode boundaries in G using ECS
    else                                          ▷ success
        Copy fully resolved tiles from B into G
    end if
end while
```

POMS retains a copy of the larger grid, keeping either the fully resolved tile per cell or an indicator that the cell is in an indeterminate state. Each round consists of choosing a block from some block choice scheduler and doing full block level resolution from some underlying block level algorithm. In this paper, we only consider Breakout Model Synthesis (BMS) [1] for the underlying block level resolution algorithm.

Each block chosen has its boundary pinned to the values as they appear in the grid, except if the block boundary falls on the grid boundary. Here, *pinning* means setting a cell's tile domain to a certain set of values and not allowing any constraint propagation to be performed on the cell. The pinned cell's tile domain can affect a neighboring cell tile but is otherwise not considered for constraint propagation. Pinning ensures that the block can integrate into the larger grid by guaranteeing the boundary conditions of the block match the related regions in the grid.

If the block and grid share a boundary, the tiles at the cell boundaries are not pinned but are subject to boundary conditions. Here, we only consider boundary conditions where a fixed tile is used when neighboring constraints would fall out of grid bounds.

Block regions are chosen by a *Block Choice Scheduler* (BCS) as referenced in Algorithm 1. From our experimentation, the BCS is problem specific and will be discussed later (Section 4).

Once a block is chosen, the block is initialized by adding the full tile domain of tile values to every cell in the block, applying setup restrictions and then running an initial constraint propagation that attempts to put the block in an arc consistent state. The setup restrictions only allow for tiles to be added, removed or pinned on an individual cell. Once setup restrictions are imposed, an attempt is made to run the constraint propagation and put the block in an arc consistent state.

If the block is successfully put in an initial arc consistent state, the block level resolution algorithm proceeds and attempts to find a fully realized block. If a fully realized block is found, it's put back into the grid and the algorithm continues on by choosing another block to process.

If the initial block level arc consistency attempt succeeded but the block level resolution algorithm failed to find a fully realized block, boundary tiles of fully resolved regions in the whole grid are probabilistically reverted to an indeterminate state. The process of probabilistically reverting boundary cell locations to indeterminate states is called *erosion*, with the *probability of erosion* being a tunable parameter that sets the probability that a cell located on a resolved boundary is reverted.

Should the initial attempt to put the block choice into an arc consistent state fail, the block region is reverted to an indeterminate state in the grid. The motivation for reverting the whole block region, as opposed to just relying on erosion, is that if a block cannot be put into an initial arc consistent state, subject to its pinned boundary values, there might be a strong contradiction that requires an aggressive back-off strategy.

Algorithm 1 gives pseudo code for the Punch Out Model Synthesis (POMS) algorithm. Figure 2 gives an overview of the POMS algorithm, highlighting a single step. Figure 3 shows a slideshow of POMS being run on the *Pill Mortal* tile set for a 64x64 grid size with a 32x32 block size.

The probability of erosion is managed by the *Erosion Choice Scheduler* (ECS) as referenced in Algorithm 1. We have found that a good choice for ECS is to increase the erosion probability by the number of failed attempts, allowing for a more aggressive erosion should block resolution become increasingly difficult. Without the erosion, block level solvers could perpetually attempt resolution on blocks with identical initial state. The erosion provides a backtracking mechanism, allowing for a change in initial state of chosen blocks and removal of implied contradictions that can occur

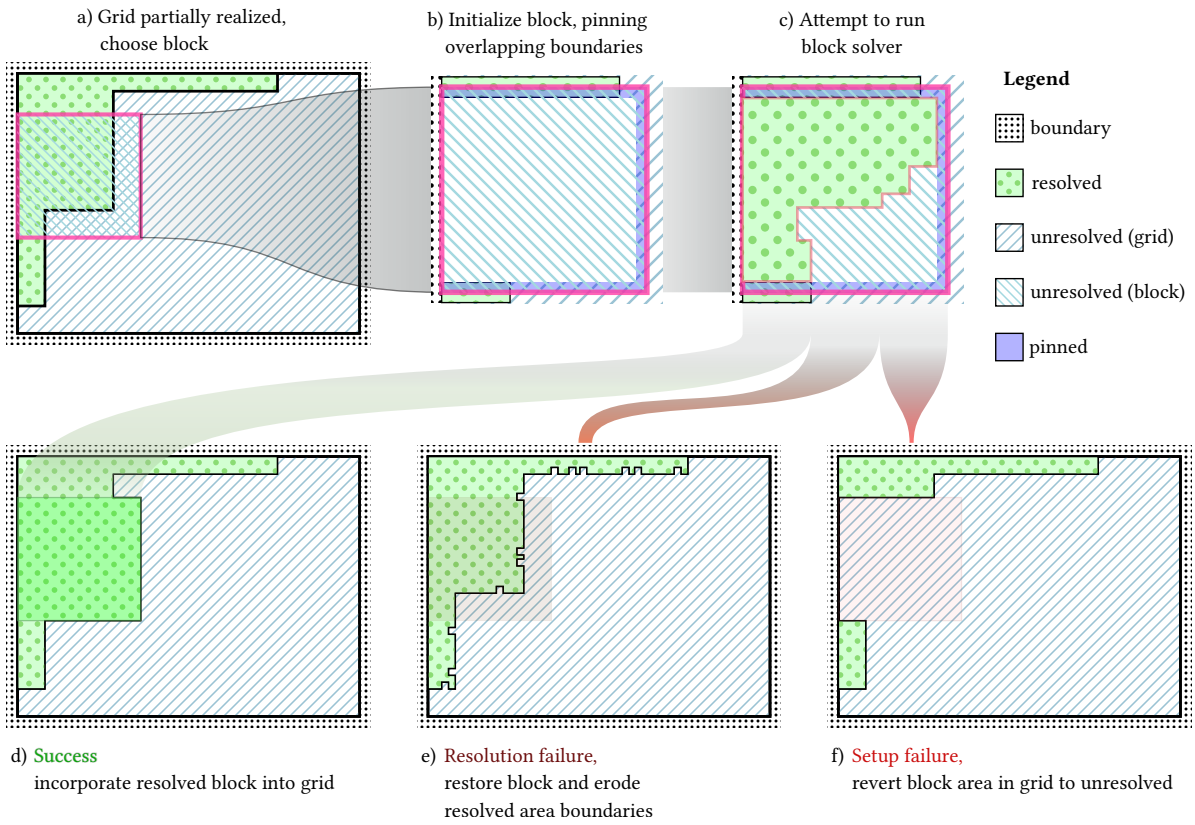

Figure 2: a) A block is chosen in the partially resolved grid, based on a block choice scheduler b) Once the block is chosen, the boundary is pinned if not on the a grid boundary and the center put into an indeterminate state. c) The block level solver attempts to find a solution for the block, with any pinned boundary restrictions d) If successful, the block is incorporated back into the grid. e) If the block solver algorithm failed to resolve, after some maximum iteration count, say, then the grid is restored to its previous state and resolved boundaries are eroded based on an erosion choice scheduler. f) If the block solver algorithm failed to start because the block could not be put into an arc consistent state given the tiles pinned on the boundary, the block area in the grid is reverted to an indeterminate state.

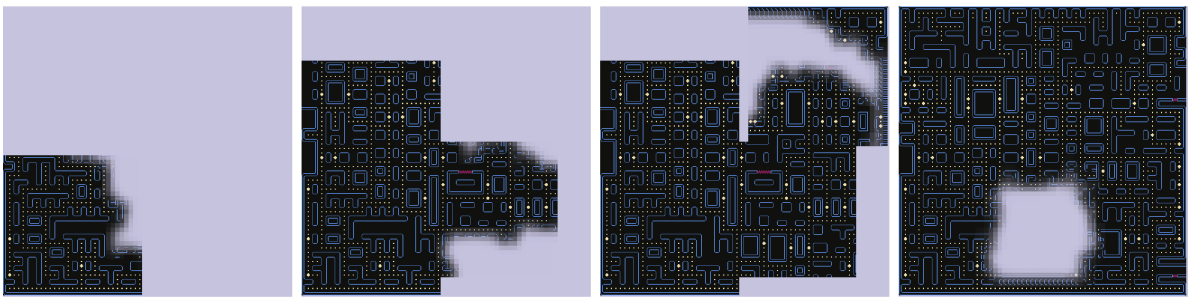

Figure 3: A slideshow of POMS run on the *Pill Mortal* tile set. The block size is 32x32 and the grid size is 64x64 with a block choice policy that chooses block centers uniformly at random from the available unresolved cell locations in the grid.

from previously resolved regions in the grid.

Maximum iteration counts can be added so that POMS or the underlying block level resolution algorithm (A) don't loop forever. The iteration counts and other checks in Algorithm 1 have been omitted for brevity.

Since the grid only keeps one value per cell and full constraint propagation is done on a block level, only a block level's worth of data need be kept in active memory. Full constraint propagation is done on an individual block level but the block size can be chosen to be much smaller than the grid, allowing block sizes to be tuned as resources allow.

3.2. Tile Arc Consistent Correlation Length

Choosing a block size for POMS is informed by the tile correlation length, as block sizes below the the tile correlation length run the risk of getting trapped in a local energy minima without the possibility of escape. As a heuristic to estimate the tile correlation length, Hoetzlein's `just_math` project [1] proposed a Tile Arc Consistent Correlation Length (TACCL).

The computation of the TACCL is done as a preprocessing step independent of the POMS algorithm. The idea is to iterate through each tile in isolation and record the maximum distance constraint propagation reaches when

resolving a center cell.

Starting from a small test grid set to an indeterminate state, the center cell is resolved to a tile and constraint propagation is performed to put the grid into an arc consistent state. The size of a bounding box that minimally encompasses every cell whose tile domain was altered during the constraint propagation is then saved. The grid is then reverted to an indeterminate state and the next tile is chosen to resolve, repeating the process and updating the maximum distances of the minimum encompassing bounding box for each tile tested.

The TACCL is the maximum extent of the saved bounding boxes from iterating through all tiles. Algorithm 2 provides pseudo-code for the TACCL calculation.

Algorithm 2 Tile Arc Consistent Correlation Length

Output: Tile Arc Consistent Correlation Length L
Input: Block Size (M_x, M_y, M_z)
Create a block B of size (M_x, M_y, M_z)
Put block B in a fully indeterminate state
$L = 1$
for tile every tile $t \in D$ **do**
  $B' = B$
  Apply initial setup restrictions to B'
  Resolve center cell, c, in B' to t
  Make B' arc consistent
  Find minimum bounding box, $bbox$,
   encompassing altered cells of B'
  if $\max_{x,y,z} bbox > L$ **then**
    $L = \max_{x,y,z} bbox$
  end if
end for
Return L

For the sake of brevity, no checks are performed in Algorithm 2 to determine if the calculated value is as large as the input block size (M_x, M_y, M_z). In such a case, either the TACCL is unbounded or the test block size is smaller than the TACCL and the test block size would need to be increased.

The TACCL is meant to estimate the correlation length of an underlying tile constraint set but can be misleading as some tile constraints will have a finite TACCL even though correlation lengths can be unbounded. An unbounded TACCL implies an unbounded correlation length but the converse is not true and a finite TACCL does not necessarily imply a finite correlation length. The *Brutal Plum* tile set, that has an unbounded correlation length but finite TACCL, displays this phenomenon and will be discussed later (Section 4).

4. Results

In this section, we highlight five tile sets that represent different aspects of the benefits and pitfalls of the Punch Out Model Synthesis (POMS) algorithm.

We start with a brief review of the automatic tile constraint creation. Though automatic tile constraint creation is a distinct topic from Constraint Based Tiling Generation (CBTG), the two are closely related as any CBTG algorithm requires a tile constraint set to run.

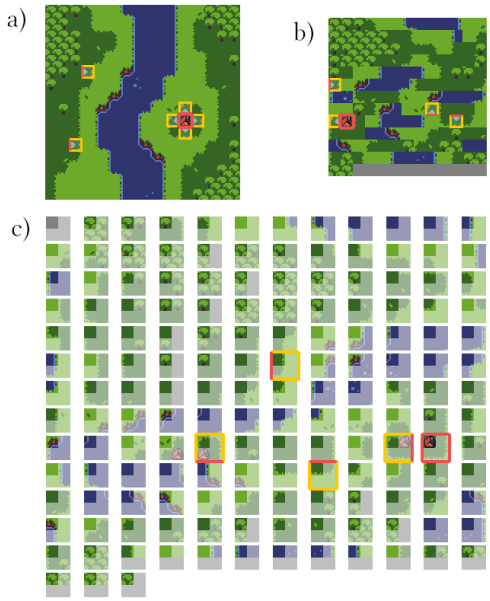

Figure 4: a) Exemplar image with a single tile and its neighbors highlighted. b) The packed image inferred tile set with the relevant tile highlighted. c) The catalog of 2x2 super tiles used to create the tile constraints from the 1x2 tile overlap, suitably rotated. The same tile highlighted in a) and b) has been highlighted here for comparison.

4.1. Automatic Tile Constraint Creation

The four 2D tile sets used in this paper had their tile constraints inferred from an exemplar image using an automated tile constraint generation method. The exemplar image is split into tiles and a sliding super tile window, of a larger size than the tile itself, is moved over the exemplar image. The super tiles are deduplicated and checked for overlapping bands to other super tiles. For every matching, overlapping band, a tile constraint is added, allowing an admissible pairing for the tiles in the appropriate dimension direction.

Figure 4 shows the exemplar image (*a*), the inferred tile set (*b*) and the complete list of super tiles for Woźniak's *Forest Micro* tile set (*c*). A single tile is highlighted in red and its neighbors are highlighted in yellow with red edges corresponding to their neighboring direction.

Figure 4 uses a tile size of 8x8 pixels, with a 2x2 super tile size. The upper left hand tile is used as the displayed representative tile, which can be seen by comparing the non dimmed tiles in the super tile list from Figure 4 (*c*) to the packed tile set image (*b*). A 1x2 overlapping band is tested for equality in each direction, suitably rotated, to find which super tiles neighbor other super tiles. An interested reader can confirm that there is a 1x2 overlap to the highlighted red super tile to each of its highlighted yellow neighbors in Figure 4.

Boundary constraints need special consideration. One option is to only allow a special "zero" tile at grid boundaries, ensuring "zero" tile constraints are added for tile pairs that fall across the edge boundaries of the exemplar image. This is the option chosen for Woźniak's *Forest Micro* tile set and can be seen in the "zero" (gray) tiles present for the super tiles listed in Figure 4 (*c*). Another option is to have *wrap around boundary conditions*, with a sliding window that wraps right or up to left or down directions, respectively,